SOFTWARE ARCHITECTURE DOCUMENT

SAD — Version 2.1.0

Identity & Access Management (IAM) Subsystem Integration

Enterprise SSO & Self-Healing RBAC Architecture

Field	Value
Project	Intervention Management Platform (UK Water Sector)
System Framework	.NET 8.0 LTS — DDD Clean Architecture
Author / Principal Architect	Harshal Yelpale
Document Version	2.1.0 (Reviewed & Enhanced)
Date of Issuance	June 2026
Classification	Internal — Restricted
Status	Production-Ready (UAT Approved)

§1 Document Control & Metadata

Revision history, technology stack, and document purpose

1.1 Revision History

Version	Date	Author	Changes
1.0.0	12-Jun-2026	H. Yelpale	Initial SSO branch formulation. Dual-login schema models established under feature_hby_SSO_12JUN26.
1.1.0	20-Jun-2026	H. Yelpale	Resolved MSAL caching race conditions. Direct header injection in UI layer. IT-group typo normalisation added.
2.0.0	26-Jun-2026	H. Yelpale	Self-healing role elevation mappings. Multi-tier deployment slot sticky configurations finalised.
2.1.0	Jul-2026	H. Yelpale	Enhanced SAD: Executive summary expanded, environment topology, App Registration detail, Graph API section, Security Considerations, Risk Register, Auto-Provisioning flow added.

1.2 Core Technology Stack

Layer	Technology	Notes
Frontend UI	ASP.NET Core MVC (.NET 8 LTS)	Cookie + OIDC dual-auth pipeline
Backend API	ASP.NET Core Web API (.NET 8 LTS)	Stateless RESTful resource server, JWT Bearer
Identity Provider	Microsoft Entra ID (Azure AD)	OpenID Connect + OAuth 2.0
Directory Integration	Microsoft Graph API	User profile, groups, manager enrichment
ORM / Data Access	Dapper + raw parameterised SQL	High-performance; no EF Core overhead
Database	Azure SQL (MS SQL Server)	Schema [IMT]; per-environment instances
Secret Management	Azure Key Vault + Managed Identity	System-assigned MI; no secrets in code
Hosting	Azure App Service (Linux Containers)	Deployment slots per environment
Caching	IMemoryCache (UI) + IDistributedCache (API)	Redis recommended for multi-instance prod
Logging	NLog	Structured log sinks; Application Insights integration
Architecture Pattern	Clean Architecture + DDD	Domain, Application, Infrastructure, Presentation layers

§2 Executive Summary & Architecture Overview

Business context, objectives, scope, and high-level design

2.1 Business Objective

The Intervention Management Platform (IMP) serves as the mission-critical logging, routing, and audit ledger for pipeline incidents, contamination boundaries, and infrastructure failures across regulated UK water utility operations. Given sector regulatory requirements (Ofwat, DWI), the platform must maintain absolute authentication precision, multi-role security enforcement, and verifiable access histories.

The primary business objective of this IAM integration initiative is to:

- Eliminate username/password credential fatigue and associated helpdesk overhead by enabling seamless Single Sign-On (SSO) through corporate Microsoft Entra ID identities.
- Align authentication with the organisation's existing Microsoft 365 tenant — the same identity users authenticate with on their company laptops and O365 applications.
- Preserve and extend the existing granular database-driven RBAC model (Roles, Permissions) without regression to existing workflows.
- Achieve zero-disruption rollout across five environments using Blue-Green deployment slot strategies.

2.2 Why Azure Entra ID SSO Was Introduced

Driver	Detail
Identity Consolidation	All staff already hold Entra ID identities via O365 licensing. A separate credential store created security risk and duplicate management overhead.
Regulatory Compliance	UK water sector compliance frameworks require auditable, centralised identity governance. Entra ID provides conditional access, MFA enforcement, and sign-in audit logs natively.
User Experience	Users opening IMP from a company device or after logging into any O365 application are authenticated transparently — no login screen is presented.
Security Posture	Removing local SQL password hashes eliminates a credential theft attack surface. Tokens are short-lived JWTs signed by Microsoft.
Operational Efficiency	Role assignments managed in Entra ID security groups propagate automatically to IMP through the self-healing sync mechanism, reducing manual DB administration.

2.3 Key Benefits

Benefit	Description
Transparent SSO	Users on company devices see no login page — they land directly on the dashboard after silent OIDC redirect.
Centralised Identity	Single source of truth for user accounts in Entra ID; IMP DB holds authorisation data only.
Self-Healing RBAC	API detects stale or default role assignments and auto-upgrades them from live Entra group

Benefit	Description
	memberships.
JIT Provisioning	New starters who have never logged into IMP are automatically provisioned on first access — no manual DB seeding required.
Dual-Auth Fallback	Feature flag (AuthenticationFeatures:EnableSSO) enables legacy cookie login for local development and emergency break-glass access.
Audit Trail	Every authentication event is logged with OID, email, role assigned, and timestamp. Entra ID provides parallel sign-in logs.
Secret-Free Code	All credentials live in Azure Key Vault; the codebase contains zero secrets.

2.4 Scope

- Authentication via Microsoft Entra ID OIDC (OpenID Connect) for all 5 deployment environments.
- Token acquisition (UI → API, UI → Graph API) using MSAL / Microsoft.Identity.Web.
- Microsoft Graph API integration: user profile, security group memberships, manager details.
- JIT user provisioning: automatic DB record creation on first SSO login.
- Self-healing RBAC sync: automatic role elevation when Entra groups are assigned after initial provisioning.
- Database-driven RBAC: Roles, Permissions, UserRoles tables remain authoritative for fine-grained permissions.
- Multi-environment Azure infrastructure: 5 Resource Groups, 5 Key Vaults, 5 SQL databases, App Service deployment slots.
- Zero-downtime Blue-Green deployment with sticky slot configuration.

2.5 Out of Scope

- Microsoft Entra ID B2C (external customer identities) — IMP serves internal staff only.
- SAML 2.0 federation — OIDC is the chosen protocol.
- Multi-tenant Entra ID configurations — single corporate tenant only.
- Role assignment UI within IMP — role assignment is managed in Entra ID portal by IT administrators.
- Guest / external user access — only corporate directory accounts are in scope.
- Mobile application authentication — IMP is a web-only platform.

§3 Auto-Provisioning Flow

JIT user creation and self-healing role sync on first and subsequent logins

3.1 Overview

Every API request passes through EntraIdRbacClaimsTransformer. This transformer executes two distinct paths based on whether the user exists in the IMP database.

3.2 Provisioning Decision Flow

```
Entra ID authenticates user → issues ID Token + Access Token
        ↓
UI: OIDC OnTokenValidated fires
  → Graph API call: GET /me/appRoleAssignments
  → Extract Entra security group names → normalise typos
  → Store in IMemoryCache (key: UI_AppRoles_{oid})
        ↓
UI → API: HTTP request with Bearer JWT + X-Entra-App-Roles header
        ↓
API: EntraIdRbacClaimsTransformer executes
  → Extract OID from JWT claim
  → Parse X-Entra-App-Roles header → extract role tokens
        ↓
  ┌──────────────────────────────────────────────────────┐
  │      DB QUERY: GetDbUserByAzureAdObjectIdAsync       │
  └──────────────────────────────────────────────────────┘
        ↓
  ┌──────────────────────┐    ┌──────────────────────────────────────────┐
  │  PATH A: User NULL   │    │  PATH B: User Exists, Role = REQUESTER   │
  │  (New User / JIT)    │    │  but header has higher-priority role     │
  └──────────────────────┘    └──────────────────────────────────────────┘
        ↓                              ↓
  INSERT [IMT].[UM_Users]         DELETE existing UserRoles
  INSERT [IMT].[UM_UserRoles]     INSERT elevated roles from header
  (Default: REQUESTER if no       (Self-healing sync triggered)
   header groups present)
        ↓                              ↓
  Reload dbUser from DB           Reload dbUser from DB
        ↓                              ↓
  ┌──────────────────────────────────────────────────────┐
  │  GetPermissionCodesByAzureAdObjectIdAsync            │
  │  → Load all permission codes for active role         │
  └──────────────────────────────────────────────────────┘
        ↓
```

```
Inject claims: userId, ClaimTypes.Role, permission[]
Set DbRbacTransformed = true (prevent double-execution)
      ↓
API controller executes with enriched ClaimsPrincipal
```

3.3 Role Priority Matrix (Single-Active-Role Enforcement)

When a user belongs to multiple Entra security groups, the UI layer resolves to one active operational role using a priority weighting algorithm:

Priority Weight	Role	Typical User Type	Access Level
1 (Highest)	ADMIN	IT / Platform Administrators	Full system access including user management
2	APPROVER	Senior Operations Staff	Approve interventions, view all records
3	SITEMANAGER	Site / Area Managers	Manage site-level interventions
4	REQUESTER	Field Operatives / Standard Staff	Create and track own interventions
5	DEVELOPER	Development Team (Non-prod only)	Extended diagnostic and debug access
99 (Lowest)	UNKNOWN	Unrecognised / unmapped groups	No access — blocked at authorization

Self-Healing Trigger Condition

Path B fires only when: (1) the user exists in DB, (2) their current DB role is REQUESTER, AND (3) the X-Entra-App-Roles header contains at least one non-REQUESTER role. This prevents unnecessary DB writes on every request while still catching the initial provisioning lag.

§4 Environment Architecture

Resource group topology, infrastructure components, and deployment slot configuration

4.1 Resource Group Topology

Infrastructure is split across two Azure Resource Groups to enforce hard blast-radius boundaries between non-production and production workloads:

```
NON-PROD Resource Group  (rg-intervention-nonprod)
├── App Service Plan (Linux P1v3)
│   ├── Slot: DEV  (dev.intervention.com)
│   │   ├── UI App Service  (intervention-ui-dev)
│   │   └── API App Service (intervention-api-dev)
│   └── Slot: UAT  (uat.intervention.com)
│       ├── UI App Service  (intervention-ui-uat)
│       └── API App Service (intervention-api-uat)
├── Azure SQL Server (sql-intervention-nonprod)
│   ├── DB: imtdb-dev
│   └── DB: imtdb-uat
└── Azure Key Vault
    ├── KV-Dev  (kv-intervention-dev)
    └── KV-UAT  (kv-intervention-uat)

PROD Resource Group  (rg-intervention-prod)
├── App Service Plan (Linux P2v3)
│   ├── Slot: STAGING  (staging.intervention.com)  ← Blue-Green staging
│   │   ├── UI App Service  (intervention-ui-staging)
│   │   └── API App Service (intervention-api-staging)
│   ├── Slot: PREPROD  (preprod.intervention.com)
│   │   ├── UI App Service  (intervention-ui-preprod)
│   │   └── API App Service (intervention-api-preprod)
│   └── Slot: PROD  (company.intervention.com)  ← Live traffic
│       ├── UI App Service  (intervention-ui-prod)
│       └── API App Service (intervention-api-prod)
├── Azure SQL Server (sql-intervention-prod)
│   ├── DB: imtdb-staging
│   ├── DB: imtdb-preprod
│   └── DB: imtdb-prod
└── Azure Key Vault
    ├── KV-Staging  (kv-intervention-staging)
    ├── KV-PreProd  (kv-intervention-preprod)
    └── KV-Prod     (kv-intervention-prod)
```

4.2 Per-Environment Configuration Matrix

Environment	URL	KV Name	SQL DB	SSO Enabled	Slot Sticky
Local (Dev PC)	https://localhost:5001	N/A (User Secrets)	LocalDB / Docker	false	N/A
DEV	https://dev.intervention.com	kv-intervention-dev	imtdb-dev	true	Yes
UAT	https://uat.intervention.com	kv-intervention-uat	imtdb-uat	true	Yes
STAGING	https://staging.intervention.com	kv-intervention-staging	imtdb-staging	true	Yes
PREPROD	https://preprod.intervention.com	kv-intervention-preprod	imtdb-preprod	true	Yes
PROD	https://company.intervention.com	kv-intervention-prod	imtdb-prod	true	Yes

4.3 Deployment Slot Sticky Settings

Every environment variable that differs between environments is marked as a Sticky Deployment Slot Setting in Azure Portal. This prevents configuration bleed when slots are swapped during Blue-Green deployments.

App Service Config Key	Value	Sticky
AuthenticationFeatures__EnableSSO	true / false per env	Yes
AzureAd__TenantId	YOUR_TENANT_ID	Yes
AzureAd__ClientId	UI_CLIENT_ID (per env)	Yes
KeyVaultUri	https://kv-intervention-{env}.vault.azure.net/	Yes
DownstreamApi__BaseUrl	https://{env}-api.intervention.com	Yes
ASPNETCORE_ENVIRONMENT	Development / UAT / Production	Yes

Linux Container Key Naming Rule

Azure App Service on Linux maps JSON colon separators using double underscores (__), NOT single underscores or colons. Example: 'AzureAd:ClientId' in appsettings.json becomes 'AzureAd__ClientId' in the Azure Portal App Settings blade. This applies to ALL nested configuration keys.

§5 Azure App Registration Configuration

UI and API registration details, permissions, scopes, and redirect URIs

5.1 UI App Registration — intervention-ui

The UI registration drives the OIDC sign-in flow and acquires tokens for both the downstream API and Microsoft Graph.

Field	Value
Registration Name	intervention-ui
Application (Client) ID	<UI_CLIENT_ID> — copy from Azure Portal after registration
Directory (Tenant) ID	<TENANT_ID> — shared across both registrations
Supported Account Types	Accounts in this organisational directory only (Single tenant)
Application Type	Web (Confidential Client)

5.1.1 Redirect URIs

Environment	Redirect URI	Logout URI
Local	https://localhost:5001/signin-oidc	https://localhost:5001/signout-oidc
DEV	https://dev.intervention.com/signin-oidc	https://dev.intervention.com/signout-oidc
UAT	https://uat.intervention.com/signin-oidc	https://uat.intervention.com/signout-oidc
STAGING	https://staging.intervention.com/signin-oidc	https://staging.intervention.com/signout-oidc
PREPROD	https://preprod.intervention.com/signin-oidc	https://preprod.intervention.com/signout-oidc
PROD	https://company.intervention.com/signin-oidc	https://company.intervention.com/signout-oidc

5.1.2 API Permissions (UI Registration)

Permission	Type	Purpose	Admin Consent
openid	Delegated (Microsoft Graph)	Base OIDC sign-in	No
profile	Delegated (Microsoft Graph)	User display name and photo	No
email	Delegated (Microsoft Graph)	User email address claim	No
offline_access	Delegated (Microsoft Graph)	Refresh token acquisition for silent renewal	No
User.Read	Delegated (Microsoft Graph)	Read signed-in user's profile via /me	No
GroupMember.Read.All	Delegated	Read user's group memberships for role	YES — Required

Permission	Type	Purpose	Admin Consent
	(Microsoft Graph)	mapping	
Directory.Read.All	Delegated (Microsoft Graph)	Read directory objects for manager lookup	YES — Required
api://<API_CLIENT_ID>/access_as_user	Delegated (intervention-api)	Acquire access token for IMP API calls	YES — Required

5.1.3 Client Secret

A client secret is generated per environment under Certificates & Secrets. Each secret is stored in the corresponding Key Vault under the key AzureAd--ClientSecret (double-dash notation for nested config mapping).

> **Admin Consent Requirement**
> GroupMember.Read.All and Directory.Read.All require a Global Administrator or Privileged Role Administrator to grant tenant-wide admin consent in Azure Portal → App Registrations → intervention-ui → API Permissions → Grant admin consent. Without this, Graph API calls for group membership will return 403 Forbidden.

5.2 API App Registration — intervention-api

The API registration acts as a resource server. It exposes a custom OAuth 2.0 scope that the UI requests when acquiring tokens to call the IMP API.

Field	Value
Registration Name	intervention-api
Application (Client) ID	<API_CLIENT_ID> — used as Audience in JWT validation
Directory (Tenant) ID	<TENANT_ID> — same as UI registration
Application ID URI	api://<API_CLIENT_ID> — set under Expose an API
Exposed Scope	access_as_user — full URI: api://<API_CLIENT_ID>/access_as_user
Authorised Client Applications	Add UI Client ID (<UI_CLIENT_ID>) with access_as_user scope pre-consented
Redirect URIs	None — API is a stateless resource server, not a web app
Client Secret	Not required for API-only registrations using JWT Bearer validation

5.2.1 JWT Audience Validation

The API validates that incoming JWT Bearer tokens contain the correct audience claim. Configuration in appsettings:

```
// appsettings.json — API project
{
  "AzureAd": {
    "Instance":  "https://login.microsoftonline.com/",
```

```
    "TenantId":  "<TENANT_ID>",
    "ClientId":  "<API_CLIENT_ID>",
    "Audience":  "api://<API_CLIENT_ID>"
  }
}
```

§6 Database Architecture — RBAC Schema

Table definitions, data dictionary, and foreign key relationships

6.1 Schema Overview

The RBAC model resides in the [IMT] SQL Server schema. The schema uses strict foreign key constraints with cascade-delete to prevent orphan access records.

```
[IMT].[UM_Users]          ← Core user accounts; AzureAdObjectId links to Entra ID
    ↑
[IMT].[UM_UserRoles]      ← Intersection: User ↔ Role (multi-role support)
    ↑
[IMT].[UM_Roles]          ← Role master: ADMIN, APPROVER, SITEMANAGER, REQUESTER,
DEVELOPER
    ↑
[IMT].[UM_RolePermissions]← Intersection: Role ↔ Permission
    ↑
[IMT].[UM_Permissions]    ← Permission master: 30-35 granular permission codes
```

6.2 Table Definitions

6.2.1 [IMT].[UM_Users]

Column	Type	Nullable	Default	Purpose
UserId	INT	NOT NULL / PK	Identity(1,1)	Internal primary key for all DB relations
AzureAdObjectId	VARCHAR(250)	NULL / Unique	NULL	Entra ID oid GUID. Populated on first SSO login. Case-insensitive search.
Username	VARCHAR(150)	NOT NULL	—	Pre-@ email prefix for legacy system alignment
Email	VARCHAR(150)	NOT NULL / Unique	—	Primary communication index; used for email-based fallback matching
DisplayName	VARCHAR(250)	NOT NULL	—	Sanitised presentation name for UI elements
IsActive	BIT	NOT NULL	1	Authorization gate; inactive users are blocked at claims transformation
CreatedAt	DATETIME	NOT NULL	GETUTCDATE()	Audit timestamp for initial provisioning

6.2.2 [IMT].[UM_UserRoles]

Column	Type	Nullable	Default	Purpose
UserId	INT	NOT NULL / PK	—	FK → UM_Users.UserId (CASCADE DELETE)
RoleId	INT	NOT NULL / PK	—	FK → UM_Roles.RoleId

Column	Type	Nullable	Default	Purpose
AssignedAt	DATETIME	NOT NULL	GETUTCDATE()	Audit timestamp for role assignment (JIT or manual)

DB Schema Change for SSO

AzureAdObjectId was added to UM_Users as part of the v1.0.0 SSO branch. Existing users created before SSO was enabled will have NULL in this column. They are matched on first SSO login by email address, and their AzureAdObjectId is backfilled automatically by the self-healing sync path.

§7 Microsoft Graph API Integration

Permissions, API endpoints, user profile mapping, and group-to-role translation

7.1 Overview

Microsoft Graph API serves two distinct purposes in the IMP SSO architecture: (1) fetching the authenticated user's Entra security group memberships for role mapping at login time, and (2) enriching the user profile with display metadata (department, manager, job title, photo) for UI presentation.

7.2 Required Permissions

Permission	Type	Endpoint Used	Purpose	Admin Consent
User.Read	Delegated	GET /me	Read signed-in user's basic profile	No
GroupMember.Read.All	Delegated	GET /me/appRoleAssignments	Read user's Entra security group memberships for role mapping	YES
Directory.Read.All	Delegated	GET /me/manager	Read manager information from the directory	YES

7.3 API Calls & Usage

7.3.1 GET /me — Basic User Profile

```
GET https://graph.microsoft.com/v1.0/me
Authorization: Bearer {graphAccessToken}

Response fields used:
  id           → Users.AzureAdObjectId  (primary identity link)
  mail         → Users.Email            (provisioning + communication)
  displayName  → Users.DisplayName      (UI presentation name)
  jobTitle     → UI profile display only (not persisted in IMP DB)
  department   → UI profile display only
```

7.3.2 GET /me/appRoleAssignments — Security Group Memberships

```
GET https://graph.microsoft.com/v1.0/me/appRoleAssignments
Authorization: Bearer {graphAccessToken}

Response processing logic (AuthenticationServiceExtensions.cs):
  1. Filter groups where principalDisplayName contains 'SG-APPCINTERVENTIONS'
  2. Further filter by environment shortcode (DEV / UAT / PROD)
  3. Normalise IT typo: replace '-REQUESTOR' with '-REQUESTER'
  4. Extract role token: last segment after '-' split
  5. Store in IMemoryCache: key = UI_AppRoles_{oid} TTL = 60 min
```

```
  6. Forward as X-Entra-App-Roles header to API

Example group names and extracted roles:
  SG-APPCINTERVENTIONS-DEV-APPROVER   → APPROVER
  SG-APPCINTERVENTIONS-DEV-REQUESTOR  → REQUESTER  (typo normalised)
  SG-APPCINTERVENTIONS-PROD-ADMIN     → ADMIN
```

7.3.3 GET /me/manager — Manager Information

```
GET https://graph.microsoft.com/v1.0/me/manager
Authorization: Bearer {graphAccessToken}

Response fields used:
  displayName  → Manager name shown in UI profile panel
  mail         → Manager email for workflow routing (future use)

Note: Returns 404 if no manager is assigned in the directory.
Handle gracefully — not all users have a manager record.
```

7.4 User Identity Mapping

Graph API Field	IMP DB Column	Purpose
id (OID)	UM_Users.AzureAdObjectId	Primary identity link — immutable, unique per user across Entra
mail	UM_Users.Email	Communication and legacy matching for pre-existing accounts
displayName	UM_Users.DisplayName	Stored on JIT provisioning; refreshable from Graph on subsequent logins
jobTitle	Not persisted	Shown in UI profile panel only; fetched live from Graph
department	Not persisted	Shown in UI profile panel only
manager.displayName	Not persisted	Shown in UI profile panel only

7.5 Entra Security Group Naming Convention

IT administrators must follow this naming convention exactly when creating Entra security groups for IMP role assignments:

```
Pattern:  SG-APPCINTERVENTIONS-{ENV}-{ROLE}

Examples:
  SG-APPCINTERVENTIONS-DEV-ADMIN
  SG-APPCINTERVENTIONS-DEV-APPROVER
  SG-APPCINTERVENTIONS-DEV-SITEMANAGER
  SG-APPCINTERVENTIONS-DEV-REQUESTER
  SG-APPCINTERVENTIONS-UAT-APPROVER
```

```
  SG-APPCINTERVENTIONS-PROD-ADMIN

ENV values: DEV | UAT | PROD  (LOCAL maps to DEV)
ROLE values: ADMIN | APPROVER | SITEMANAGER | REQUESTER | DEVELOPER

Known IT Typo: REQUESTOR (with O) is normalised to REQUESTER (with E)
at the authentication gateway — see AuthenticationServiceExtensions.cs
```

§8 UI Application Architecture — MVC Gateway

Authentication pipeline, token handler, claims transformer

8.1 Authentication Pipeline Components

Component	File	Responsibility
Dual-Auth Setup	AuthenticationServiceExtensions.cs	Registers OIDC + Cookie auth. OnTokenValidated fires Graph lookup and seeds role cache.
Outbound Token Handler	AuthTokenHandler.cs	DelegatingHandler that injects Bearer JWT and X-Entra-App-Roles header on all API calls.
Claims Transformer	UiRbacClaimsTransformer.cs	Calls /api/me, receives UserProfileDto, injects DB roles and permissions as claims. Short-lived cache for fallback profiles.

8.2 Feature Flag: EnableSSO

The dual-auth pipeline is controlled by a configuration flag. This allows local developer builds to run with traditional cookie auth while all deployed environments use SSO:

```
// Local: appsettings.Development.json
{ "AuthenticationFeatures": { "EnableSSO": false } }

// Azure App Service (all deployed envs): Sticky App Setting
AuthenticationFeatures__EnableSSO = true
```

8.3 Stale Cache Protection

The UiRbacClaimsTransformer detects if the API returns a fallback REQUESTER-only profile (indicating the self-healing sync has not yet completed) and sets a 5-second cache TTL instead of the standard 60 minutes. This ensures the next request re-fetches the upgraded profile:

```
// FIX 2: Stale Cache Lifespan Guard
bool isFallback = profile.Roles != null &&
    profile.Roles.Contains("REQUESTER") && profile.Roles.Count == 1;

if (isFallback) {
    cacheOptions.SetAbsoluteExpiration(TimeSpan.FromSeconds(5));
    // Forces retry on next request — picks up elevated role from Path B sync
} else {
    cacheOptions.SetAbsoluteExpiration(TimeSpan.FromMinutes(60));
}
```

§9 Backend API Architecture — Resource Server

JWT validation, claims transformer, Dapper repository

9.1 JWT Bearer Validation

The API uses Microsoft.Identity.Web to validate incoming JWT Bearer tokens. Validation checks: issuer (iss), audience (aud), signature (using Microsoft's public keys), and token expiry.

```
// Program.cs — API
builder.Services.AddMicrosoftIdentityWebApiAuthentication(configuration, "AzureAd");

// Audience validation: token aud claim must match api://<API_CLIENT_ID>
// Issuer validation:   token iss claim must match
https://login.microsoftonline.com/<TENANT_ID>/v2.0
```

9.2 EntraIdRbacClaimsTransformer

Executed on every authenticated API request. Reads the X-Entra-App-Roles custom header injected by the UI, queries the DB for the user record, executes JIT provisioning (Path A) or self-healing sync (Path B), then loads all permission codes and injects them as claims.

9.3 AccountRepository (Dapper)

Method	SQL Operation	Transactional
GetDbUserByAzureAdObjectIdAsync	SELECT from UM_Users WHERE AzureAdObjectId = @OID	No
ProvisionEntraIdUserAsync	INSERT UM_Users + INSERT UM_UserRoles (with role lookup)	Yes — Rollback on failure
UpdateUserRolesFromEntraIdAsync	DELETE UM_UserRoles + INSERT new roles	Yes — Rollback on failure
GetPermissionCodesByAzureAdObjectIdAsync	JOIN UM_UserRoles → UM_RolePermissions → UM_Permissions	No

§10 Security Considerations

Token security, session management, cookie hardening, and threat mitigations

10.1 Secret Management

Secret	Storage Location	Access Method	Rotation Policy
AzureAd:ClientSecret (UI)	Azure Key Vault (per env)	System-Assigned Managed Identity	Rotate every 12 months or on suspected compromise
SQL Connection String	Azure Key Vault (per env)	System-Assigned Managed Identity	Rotate on staff offboarding or quarterly
Redis Connection String	Azure Key Vault (per env)	System-Assigned Managed Identity	Rotate every 12 months
Graph API Token	In-memory only (IMemoryCache)	MSAL token acquisition at runtime	TTL: 60 minutes; refreshed silently via offline_access

Zero-Secret Policy

The application codebase and source control contain zero credentials, connection strings, or API keys. All secrets are pulled from Azure Key Vault at application startup using Managed Identity. No developer has direct access to production Key Vault secrets.

10.2 Token Lifetime & Expiry

Token Type	Default Lifetime	IMP Configuration	Renewal Mechanism
ID Token (OIDC)	60–90 minutes	Not extended	Silent OIDC redirect on expiry
Access Token (API)	60–90 minutes	Not extended	MSAL acquires silently using refresh token
Refresh Token	Up to 90 days (rolling)	Not extended	Used by MSAL for silent token renewal; revoked on logout
RBAC Cache (UI)	60 minutes (standard)	5 seconds (fallback profile)	Re-fetched on next request after expiry
Cookie Session	Session-length by default	HttpOnly + Secure + SameSite=Lax	Re-authenticated via OIDC on browser restart

10.3 Cookie Security Configuration

```
// AuthenticationServiceExtensions.cs
options.Cookie.Name     = "InterventionsAuthCookie";
options.Cookie.HttpOnly = true;     // Prevents JavaScript access (XSS protection)
options.Cookie.SecurePolicy = CookieSecurePolicy.Always;  // HTTPS only
options.Cookie.SameSite = SameSiteMode.Lax;  // CSRF protection for top-level nav
options.Cookie.Path     = "/";
```

10.4 CSRF Protection

ASP.NET Core MVC automatically includes Anti-Forgery Token validation on all POST/PUT/DELETE form submissions via the [ValidateAntiForgeryToken] attribute and the @Html.AntiForgeryToken() helper in Razor views. OIDC state parameters provide additional CSRF protection during the authentication redirect flow.

10.5 XSS Protection

- All Razor view outputs are HTML-encoded by default — @Model.Value never injects raw HTML.
- Content Security Policy (CSP) headers should be configured to restrict script sources to trusted origins only.
- X-Content-Type-Options: nosniff and X-Frame-Options: DENY headers must be set.
- User-supplied strings (DisplayName, Email) are never executed as code paths.

10.6 Token Replay Protection

- Entra ID issues short-lived access tokens (60–90 min). Replay window is minimised.
- JWT nonce claim (nonce) in ID tokens prevents replay of authentication responses.
- OIDC state parameter prevents CSRF during the authorisation code flow.
- PKCE (Proof Key for Code Exchange) is enforced by Microsoft.Identity.Web on all OIDC flows.

10.7 Logout & Session Termination

```
// Full logout: clears cookie AND signs out of Entra ID
return SignOut(
    new AuthenticationProperties { RedirectUri = "/" },
    CookieAuthenticationDefaults.AuthenticationScheme,
    OpenIdConnectDefaults.AuthenticationScheme
);

// This sends the user to Entra ID's /logout endpoint,
// invalidating their Entra session. Prevents SSO re-authentication
// without re-entering credentials if MFA is enforced.
```

10.8 Conditional Access & MFA

Microsoft Entra ID Conditional Access Policies are enforced at the tenant level by the IT/Security team. IMP does not need to implement MFA — Entra ID enforces it transparently before issuing tokens. Conditional Access can enforce MFA for all IMP logins without any code changes.

§11 Risks & Mitigation Register

Identified technical, operational, and configuration risks with mitigations

#	Risk	Likelihood	Impact	Mitigation
R1	Graph API permission missing or admin consent not granted	Medium	HIGH — All users fail to load roles; fallback to REQUESTER	Pre-register GroupMember.Read.All and Directory.Read.All. Require IT admin to grant tenant consent during deployment. Validate in staging before PROD.
R2	Incorrect Redirect URI configured in App Registration	Medium	HIGH — AADSTS50011 error; all logins fail	Document all 6 redirect URIs (5 envs + local). Validate after every App Registration change. Use automated smoke test post-deployment.
R3	Azure Key Vault unavailable at application startup	Low	HIGH — App fails to start; all environments down	App Service has Managed Identity retry logic. Implement health-check endpoint. Key Vault has 99.9% SLA. Consider caching critical secrets in App Service App Settings as last-resort fallback.
R4	User not found in DB on first SSO login	Certain (by design)	LOW — Handled by JIT provisioning Path A	JIT provisioning is the designed response. Default role (REQUESTER) assigned. Self-healing Path B upgrades role on next request if Entra groups are present.
R5	Slot swap scrambles environment configuration	Low	HIGH — PROD app connects to wrong DB	All environment-specific settings marked as Sticky Deployment Slot Settings. Post-swap smoke tests validate DB connectivity and Key Vault access.
R6	Entra security group typo (REQUESTOR vs REQUESTER)	High (observed)	MEDIUM — User gets wrong or no role	Inline normalisation in AuthenticationServiceExtensions.cs replaces -REQUESTOR with -REQUESTER. Known typos should be documented and IT notified to fix at source.
R7	MSAL token cache miss on	Medium	MEDIUM	Replace IMemoryCache with IDistributedCache backed by Redis (Azure

#	Risk	Likelihood	Impact	Mitigation
	multi-instance App Service		— User re-authenticated mid-session	Cache for Redis). One Redis instance per environment.
R8	Self-healing sync fires on every request (DB thrash)	Low	MEDIUM — Excess DB writes under load	Path B only fires when DB role = REQUESTER AND header contains elevated role. Once DB is updated, Path B does not re-trigger. Monitor with NLog metrics.
R9	Refresh token revocation not propagated to IMP	Low	MEDIUM — User continues active session after IT revokes access	MSAL will fail to refresh the access token on next silent acquire attempt. API returns 401. UI redirects to login. Session terminates within one access token lifetime (~60 min).
R10	Graph API rate limiting	Low	LOW — Retry degrades login performance	Graph calls occur only once per login (OnTokenValidated), cached for 60 minutes. Rate limiting is unlikely with normal user volumes.

§12 Cloud Infrastructure & Zero-Downtime Deployment

Key Vault, Managed Identity, Blue-Green slot strategy

12.1 Key Vault Secret Reference Map

Key Vault Secret Name	Maps To Config Key	Used By
AzureAd--ClientSecret	AzureAd:ClientSecret	UI App — OIDC confidential client
ConnectionStrings--DefaultConnection	ConnectionStrings:DefaultConnection	API App — Dapper SQL connection
Redis--ConnectionString	Redis:ConnectionString	UI + API — IDistributedCache
DownstreamApi--BaseUrl	DownstreamApi:BaseUrl	UI App — API base URL per env

Double-Dash Mapping Rule

Azure Key Vault does not support colons in secret names. .NET Configuration maps double-dash (--) to colon (:) automatically. Example: Key Vault secret 'AzureAd--ClientSecret' becomes 'AzureAd:ClientSecret' in the configuration system at runtime. This is distinct from App Service's double-underscore (__) rule for nested JSON keys.

12.2 Managed Identity Setup

- Each App Service (UI and API, per environment) has a System-Assigned Managed Identity enabled.
- Each Managed Identity is granted the Key Vault Secrets User role on its corresponding Key Vault.
- No service principal credentials are needed — Azure handles token exchange internally.
- DefaultAzureCredential() in code automatically uses Managed Identity in Azure and developer credentials locally.

12.3 Zero-Downtime Blue-Green Swap

```
Deployment Flow:

1. Deploy new build to STAGING slot
   → Key Vault connectivity validated
   → DB migration scripts executed
   → Health check endpoint returns 200

2. Azure performs atomic network routing switch
   → STAGING slot becomes PRODUCTION
   → Old PRODUCTION becomes STAGING (kept for rollback)
   → Sticky settings remain bound to each slot

3. Post-swap smoke tests execute automatically
   → Authentication flow tested
   → API health check validated

4. If error spike detected within 5 minutes:
```

```
→ Automatic rollback swap reverses routing
→ Old container back online in < 30 seconds
```

§13 Branching Strategy

Feature isolation, progressive verification, and release pipeline

13.1 Branch Isolation

Branch	Purpose	Lifetime
main	Production-stable code. No direct commits.	Permanent
dev	Integration branch. All feature branches merge here first.	Permanent
feature_hby_SSO_12JUN26	Complete SSO implementation isolation. Both UI and API changes.	Deleted post-merge
hotfix/*	Emergency production fixes.	Short-lived

13.2 Progressive Verification Path

```
feature_hby_SSO_12JUN26
    ↓  Local verification (EnableSSO=false, cookie auth)
    ↓  Local SSO test (EnableSSO=true, localhost redirect URI)
    ↓  Push to DEV slot
    ↓  DEV smoke test (SSO flow, RBAC claims, API calls)
    ↓  PR review → merge to dev
    ↓  Deploy to UAT slot
    ↓  UAT regression tests + stakeholder sign-off
    ↓  Deploy to STAGING (Blue-Green)
    ↓  Staging warmup + automated tests
    ↓  Slot swap → PRODUCTION
    ↓  Post-swap monitoring (15 minutes)
```

§A Appendix

Glossary, abbreviations, and quick reference

A.1 Glossary

Term	Definition
Entra ID	Microsoft's cloud identity platform (formerly Azure Active Directory). Manages users, groups, and application registrations.
OIDC	OpenID Connect — identity layer on top of OAuth 2.0. Used by IMP for authentication.
JWT	JSON Web Token — compact, signed token format used for API authentication (Bearer tokens).
MSAL	Microsoft Authentication Library — handles token acquisition, caching, and silent renewal.
OID	Object Identifier — the immutable, unique identifier for a user in Entra ID (claim name: oid).
JIT Provisioning	Just-In-Time provisioning — creating a user DB record automatically on their first login.
RBAC	Role-Based Access Control — permissions granted based on the user's assigned role.
Blue-Green	Deployment strategy using two identical slots; traffic is switched atomically with zero downtime.
Sticky Setting	Azure App Service configuration value that remains bound to a deployment slot and does not swap with the code.
Self-Healing Sync	Automatic detection and correction of stale/default role assignments by comparing DB state to live Entra group membership.
Managed Identity	Azure service that provides an automatically managed identity for App Services, eliminating the need for stored credentials.
Fallback Profile	A user profile returned by the API with only the REQUESTER role when JIT provisioning has just run and elevated roles are not yet in the DB.

A.2 Key File Index

File	Project	Purpose
AuthenticationServiceExtensions.cs	UI (InterventionsManagementUI)	Dual-auth pipeline registration + Graph group fetch on OIDC token validation
AuthTokenHandler.cs	UI (Infrastructure)	DelegatingHandler: injects Bearer token + X-Entra-App-Roles header
UiRbacClaimsTransformer.cs	UI (AppFrontend)	IClaimsTransformation: calls /api/me, builds ClaimsPrincipal with DB RBAC
EntraIdRbacClaimsTransformer.cs	API (Infrastructure.Security)	IClaimsTransformation: JIT provisioning + self-healing sync + claims injection
AccountRepository.cs	API (Infrastructure.Repositories)	Dapper CRUD for UM_Users and UM_UserRoles

File	Project	Purpose
Program.cs (UI)	UI	Registers dual-auth, IMemoryCache, HttpClients, Key Vault
Program.cs (API)	API	Registers JWT Bearer, IDistributedCache (Redis), Key Vault, CORS